

NATURAL LANGUAGE PROCESSING - S2-25_AIMLCZG530

Lab-1: Vector Embeddings and Neural Language Models

The lab experiments should be conducted on Virtual Labs

Before the lab: setup checklist

Students must complete this before coming to the live session:

- Install Python 3.x and Jupyter Notebook / JupyterLab or have access to Google Colab.
- Install required libraries:
 - pandas, scikit-learn, gensim, tensorflow (or keras), nltk, matplotlib, seaborn.
- Download the following resources and keep them in a single folder or upload them to Colab:
 - Dataset 1 (spam SMS): Kaggle “Spam Text Message Classification” (SPAM-Data.csv or equivalent) - <https://www.kaggle.com/datasets/team-ai/spam-text-message-classification>
 - Dataset 2 (movie reviews): Kaggle IMDB sentiment dataset in CSV format (IMDB_Dataset.csv or similar) - <https://www.kaggle.com/datasets/columbine/imdb-dataset-sentiment-analysis-in-csv-format>
 - GloVe embeddings: glove.6B.100d.txt from the Stanford GloVe page <https://nlp.stanford.edu/projects/glove/>
 - Dataset 3 (Sherlock Homes): Kaggle next word prediction dataset (text file) - <https://www.kaggle.com/datasets/ronikdedhia/next-word-prediction>
- Open the starter notebooks and run the first cell to confirm imports work (no need to train any models yet).

Learning goals

By the end of this lab you should be able to:

1. Build frequency-based text representations (Count Vectorizer, TF-IDF) and train a simple classifier.
2. Train prediction-based embeddings (Word2Vec – CBOW and Skip-gram) and interpret word similarities.
3. Use pre-trained GloVe vectors and compare their neighbors with Word2Vec.
4. Implement simple neural language models for:
 - next-word prediction (LSTM language model)
 - Next-Word Prediction with Hugging Face

During the live session, you will run and modify code; this worksheet is for preparation and to guide your experiments.

Task 1 – Frequency-based embeddings

1A. Count Vectorizer – SMS spam classification

Dataset: Kaggle spam SMS dataset (short messages labeled ham / spam).

In-lab steps (you will do this live):

1. Load the CSV and inspect a few rows (head(), label counts).
2. Split into train/test sets.
3. Use CountVectorizer to convert messages into bag-of-words vectors.
4. Train a simple classifier (Multinomial Naive Bayes).
5. Compute accuracy and F1 on the test set.

Modify code:

Change:

- max_features (e.g., 500, 1000, 2000)
- ngram_range (e.g., (1,1) vs (1,2))
- stop_words (None vs 'english')
and record the accuracy and F1 for at least three different settings in a small table.

1B. TF-IDF – Improved representation

Dataset: same spam SMS dataset.

In-lab steps:

1. Replace CountVectorizer with TfidfVectorizer using similar parameters (same max_features, ngram_range, stop_words).
2. Train the same classifier type as in 1A.
3. Compare accuracy/F1 between Count and TF-IDF.
4. Examine some misclassified examples for both models.
5. Print the TF-IDF vector for one message and list its highest-weight words.

Task 2 – Prediction-based embeddings

2A. Word2Vec – word similarities

Dataset: IMDB movie reviews dataset.

In-lab steps:

1. Load the CSV and sample a subset (e.g., 2,000 reviews for speed).
2. Preprocess: lowercase, tokenize, remove punctuation and stopwords, optionally lemmatize.
3. Train a Word2Vec model (Skip-gram or CBOW) on the tokenized reviews.
4. For a chosen list of words (e.g., movie, film, actor, story, good, bad), list the top-k most similar words using cosine similarity.
5. Visualize a small set of word vectors using PCA (2D scatterplot with labels).

2B. GloVe – global co-occurrence

Resource: glove.6B.100d.txt pre-trained embeddings.

In-lab steps:

1. Load GloVe vectors into a Python dictionary: word -> vector.

2. For the same query words used in Word2Vec, retrieve top-k nearest neighbors using cosine similarity.
3. Compare the neighbor lists from GloVe and your Word2Vec model side by side.

Task 3 – Neural language models

3A. Next word prediction - RNN

3A-1. Student activity

Next word prediction – LSTM language model

Dataset: Sherlock Homes text dataset

In-lab steps:

1. Load the corpus and extract text into a list.
2. Tokenize and build a vocabulary; turn text into integer sequences.
3. Create training samples: input sequence → next word (as in the demo code).
4. Build a small LSTM model for next-word prediction.
5. Train for a few epochs and compute validation loss and perplexity.
6. For several prompts (e.g., Once upon a time, The little robot, In the dark night), generate the top-3 next-word candidates.

3B. Next-Word Prediction with Hugging Face vs LSTM

Use a pretrained Hugging Face language model to generate next-word predictions for given prompts, and compare its output with the LSTM-based neural language model you built earlier.

Instructions for students

1. Choose at least three prompts from the list below, or create your own:
 - Once upon a time
 - The little boy
 - In the dark forest
2. For each prompt, do the following:

- Generate the top-3 next-word candidates.
- Generate a short continuation of about 5–10 words.